A Project Synopsis
on

SMART TIMETABLE SCHEDULER (EDUSCHEDULER)

Submitted in partial fulfilment of the Requirements For the award of the degree of

Bachelor of Technology
in
Computer Science & Engineering

Submitted To:
Mrs. Stuti Mehla
Designation:
Assistant professor

Submitted By:
Ankit Kumar Sah (2823205)
Yash (2823203)
Vipul Gupta (2823248)
Golu Gupta (2823194)

Panipat Institute of Engineering & Technology, Samalkha, Panipat

Affiliated to

Kurukshetra University Kurukshetra, India

(2025-2026)

PANIPAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

Department of Computer Science and Engineering

UNDERTAKING

Yash, Ankit Kumar Sah, Vipul gupta, Golu Roll No:2823203, 2823205, 2823248, 2823194 B. Tech. (Semester- VI) of the Panipat Institute of Engineering & Technology, Samalkha (Panipat) hereby declare that I am the whole sole responsible for the timely submission of the Project I. I also hereby declare that I am whole sole responsible for the project work should be done as per project guideline issued by the department, if any shortcoming will be found in my work regarding timely submission and quality of work, the department will have full authority to reject my work at any point of time and also have full authority to deduct my marks.

Place: Panipat Institute of Engineering and Technology

Name of Student:

Yash (2823203)
Ankit Kumar Sah(2823205)
Vipul Gupta (2823248)
Golu Gupta (2823194)

Sign:

Table of content

Serial no.	topic	Page no.
i	undertaking	2
1.	Abstract	4
2.	Introduction	5-6
3	Literature reviews	7-8
4	Detail of Proposed System • Present State • After Implementation of project • Flow chart of Proposed System • Please describe the type of system being developed. • Who is system being developed for? • What will be the main challenge for you in building the system?	8-11
5	Scope	12
6	Significance of Project	13-14
7	Tools & Technology Used	15
8	Time Frame required for various stages of project implementation	16

ABSTRACT

EduScheduler is an intelligent academic timetable management and automation platform designed to simplify and optimize the scheduling process within educational institutions. Traditional timetable creation is often manual, time-consuming, and prone to conflicts. EduScheduler addresses these challenges by automating schedule generation through constraint-based optimization techniques.

The system intelligently generates **conflict-free timetables** while ensuring efficient allocation of faculty, classrooms, and academic sections. It incorporates a secure, role-based access system that provides dedicated dashboards for administrators, teachers, and students. This ensures structured control, personalized access, and improved usability.

With a robust backend architecture and an interactive frontend interface, EduScheduler enhances scheduling efficiency, minimizes human errors, and centralizes academic data management into a single, scalable platform.

-Key Features

Automated Timetable Generation
Uses Google OR-Tools (Constraint Programming) to automatically generate optimized, conflict-free academic timetables.

Conflict Detection Engine
Prevents faculty double-booking, room clashes, and section overlaps through rule-based constraint validation.

Role-Based Authentication
Secure JWT-based authentication system supporting Admin, Teacher, and Student portals with restricted access control.

Dynamic Timetable Visualization
Displays structured weekly timetable grids with subject, faculty, and room information in a clean and user-friendly interface.

Department & Year Filtering
Allows filtering of timetables by department, academic year, and section for easy data retrieval.

SQLite Database Integration
Uses relational database design for structured storage of users, departments, faculty, rooms, and schedule data.

CSV Import/Export Support
Enables bulk data import and timetable export for backup and external usage.

Admin Dashboard Management
Provides administrative control to manage users, generate schedules, and monitor academic data efficiently.

Modular Backend Architecture
Built with Flask and organized using clean routing, service layers, and modular design principles.

.

INTRODUCTION

Timetable scheduling is a fundamental operational activity in educational institutions that involves systematic allocation of subjects, faculty members, classrooms, and time slots while satisfying institutional constraints. As academic institutions grow in size and complexity, manual timetable preparation becomes increasingly inefficient and error-prone. The scheduling process must consider multiple constraints such as faculty availability, classroom capacity, subject distribution across the week, avoidance of overlapping sessions, and compliance with institutional working hours. Managing these interdependent constraints manually often results in scheduling conflicts, uneven workload distribution, and excessive administrative effort.

From a computational perspective, timetable generation is a combinatorial optimization problem and can be modeled as a Constraint Satisfaction Problem (CSP). In such problems, variables must be assigned values from a domain in a way that satisfies a set of constraints. In the context of academic scheduling, each time slot can be considered a variable, while combinations of subject, faculty, and classroom represent possible domain values. Constraints include preventing faculty double-booking, avoiding room allocation conflicts, ensuring required weekly subject hours are met, and maintaining fairness in workload distribution.

Traditional approaches to timetable creation typically rely on spreadsheets or semi-automated tools where administrators manually assign classes and resolve conflicts iteratively. While this method may work for small institutions, it becomes highly inefficient as the number of departments, courses, and faculty increases. Moreover, manual systems lack scalability, adaptability to dynamic changes, and optimization capabilities.

The Smart Timetable Scheduler is proposed as a web-based automated system that addresses these limitations by integrating constraint-based scheduling logic with modern full-stack web technologies. The system is designed using a client-server architecture, where the frontend interface is developed using ReactJS with shadcn/ui to provide a responsive and user-friendly administrative dashboard. The backend is implemented using Flask, a lightweight Python-based web framework, which handles business logic, scheduling algorithms, and API communication. Database operations are managed through SQLAlchemy ORM, ensuring structured data persistence and abstraction from raw SQL queries.

The core functionality of the system lies in its scheduling engine, which applies constraint validation and allocation logic to generate conflict-free timetables. The engine systematically assigns time slots using backtracking or greedy-based strategies while dynamically checking constraints at each allocation step. If a violation occurs, the algorithm reassigns values until a feasible schedule is obtained. This approach ensures systematic exploration of possible allocations while maintaining computational efficiency.

In addition to scheduling logic, the system includes modules for faculty management, subject management, classroom allocation, and CSV-based data import/export. These modules ensure that administrators can easily manage institutional data and generate timetables dynamically without technical complexity. The use of RESTful APIs ensures modular communication between frontend and backend components, enhancing maintainability and scalability.

The development of this project demonstrates the practical application of several core Computer Science and Engineering concepts, including:

Data Structures and Algorithms (constraint handling, backtracking)
Database Management Systems (relational modeling, ORM usage)
Software Engineering (modular design, layered architecture)
Web Technologies (frontend-backend integration).

.

-*Objectives of the Project:*

The primary objectives of the EduScheduler system are as follows:

• Automated Timetable Generation
To develop an intelligent scheduling system that automatically generates conflict-free academic timetables using constraint programming techniques, eliminating the need for manual planning.

• Conflict Prevention and Resource Optimization
To ensure efficient allocation of faculty, classrooms, and academic sections while preventing scheduling conflicts such as faculty double-booking, classroom overlaps, and section clashes.

• Secure Role-Based Access Control
To implement a secure authentication mechanism using JWT that provides role-based access to administrators, teachers, and students, ensuring data privacy and controlled system usage.

• Improved User Experience and Accessibility
To design a clean, structured, and user-friendly interface that allows users to easily view, manage, and interact with timetable data.

• Efficient Data Management and Backup Support
To utilize a relational database system for structured data storage and provide CSV import/export functionality for efficient bulk data handling and backup.

• Scalable and Maintainable System Architecture
To develop a modular, scalable, and cloud-deployable backend using modern web development practices, ensuring long-term maintainability and future expansion.

• Enhanced Institutional Efficiency
To reduce administrative workload, improve scheduling accuracy, and streamline academic timetable management processes.
.

LITERATURE REVIEW

Existing Systems & Their Limitations:

1. Manual Timetable Creation

In many educational institutions, timetable preparation is still performed using manual methods such as spreadsheets (e.g., Microsoft Excel) or even paper-based planning. This traditional approach requires administrators to consider numerous variables, including faculty availability, classroom capacity, subject requirements, and departmental constraints. Managing these factors manually becomes increasingly difficult as the size and complexity of the institution grow.

This process is not only highly time-consuming but also extremely prone to human errors. Even minor mistakes can result in faculty scheduling conflicts, classroom double-booking, or uneven workload distribution. Furthermore, making adjustments to an existing timetable often requires extensive rework, which further increases administrative burden. The lack of automation and intelligent validation leads to inefficient time utilization and reduced operational efficiency.

---

2. Basic Academic Management Systems

Some educational institutions utilize basic academic management systems that provide limited timetable generation functionality. These systems often rely on predefined templates or simple rule-based logic without incorporating advanced optimization techniques. As a result, they generate static timetables that do not dynamically adapt to institutional constraints.

These systems typically lack real-time conflict detection and intelligent validation mechanisms. Administrators are still required to manually verify the generated timetables to identify conflicts such as overlapping classes, faculty double assignments, or room allocation issues. This partial automation fails to fully eliminate manual intervention and does not significantly improve scheduling efficiency.

Additionally, these systems offer limited flexibility and may not support dynamic changes, making timetable modifications difficult and inefficient.

---

3. Commercial ERP Solutions

Enterprise Resource Planning (ERP) systems provide comprehensive academic management solutions, including timetable scheduling, student management, and faculty administration. These systems often include advanced scheduling features and integrated modules designed for large-scale institutions.

However, ERP solutions have several significant drawbacks. They are typically expensive to purchase, implement, and maintain, making them financially inaccessible for small and medium-sized educational institutions. Furthermore, ERP systems are often complex and require specialized training for administrators and staff to operate effectively.

Customization of ERP systems to meet specific institutional requirements can be difficult and may require additional development costs. This complexity and high cost make ERP solutions impractical for institutions seeking simple, flexible, and affordable timetable management systems.

---

4. Non-Optimized Automated Systems

Some automated timetable generation systems attempt to reduce manual effort but lack proper constraint-based optimization techniques. These systems often generate schedules without fully validating essential constraints such as faculty availability, classroom allocation, and workload balance.

As a result, the generated timetables may contain serious issues, including faculty overload, classroom conflicts, or inefficient distribution of academic sessions. These systems fail to ensure optimal utilization of resources and may still require manual correction.

The absence of intelligent constraint-solving algorithms significantly reduces scheduling accuracy and overall system reliability. Without proper optimization, these automated systems cannot effectively address the complexities involved in academic timetable scheduling.

.

Why This System?

I Automates timetable generation using constraint-based optimization (Google OR-Tools) to eliminate scheduling conflicts.

I Provides secure, role-based access for administrators, teachers, and students through JWT authentication.

I Prevents faculty, classroom, and section clashes using intelligent backend validation logic.

I Offers structured, relational database management using SQLite for reliable and consistent data handling.

I Supports CSV import/export for bulk data management and backup purposes.

I Lightweight, scalable, and modular architecture built using Flask, ensuring maintainability and future extensibility.

DETAIL OF PROPOSED SYSTEM

a) Present State (Existing System)

- Timetables are often created manually using spreadsheets or paper-based systems.
- Scheduling multiple departments, faculty, and rooms leads to frequent clashes and errors.
- **Conflict detection (faculty, room, section overlap) is performed manually and is time-consuming.**
- Modifications require re-editing the entire timetable structure.
- Modifications require re-editing the entire timetable structure.

b)After Implementation (Future System)

- Timetables are generated automatically using constraint-based optimization (Google OR-Tools).
- Faculty, room, and section conflicts are eliminated through intelligent rule validation
- Administrators can manage departments, faculty, and schedules from a centralized dashboard.
- Students and teachers can securely access personalized timetables through role-based authentication.
- Data is stored in a structured relational database (PostgreSQL/SQLite) ensuring consistency and reliability.
- CSV import/export enables bulk data handling and easy backup.

c)Flowchart of Proposed System:

Admin Input —► Add departments, faculty, rooms, subjects, and constraints.

Data Storage —► Store structured data in relational database (PostgreSQL/SQLite).

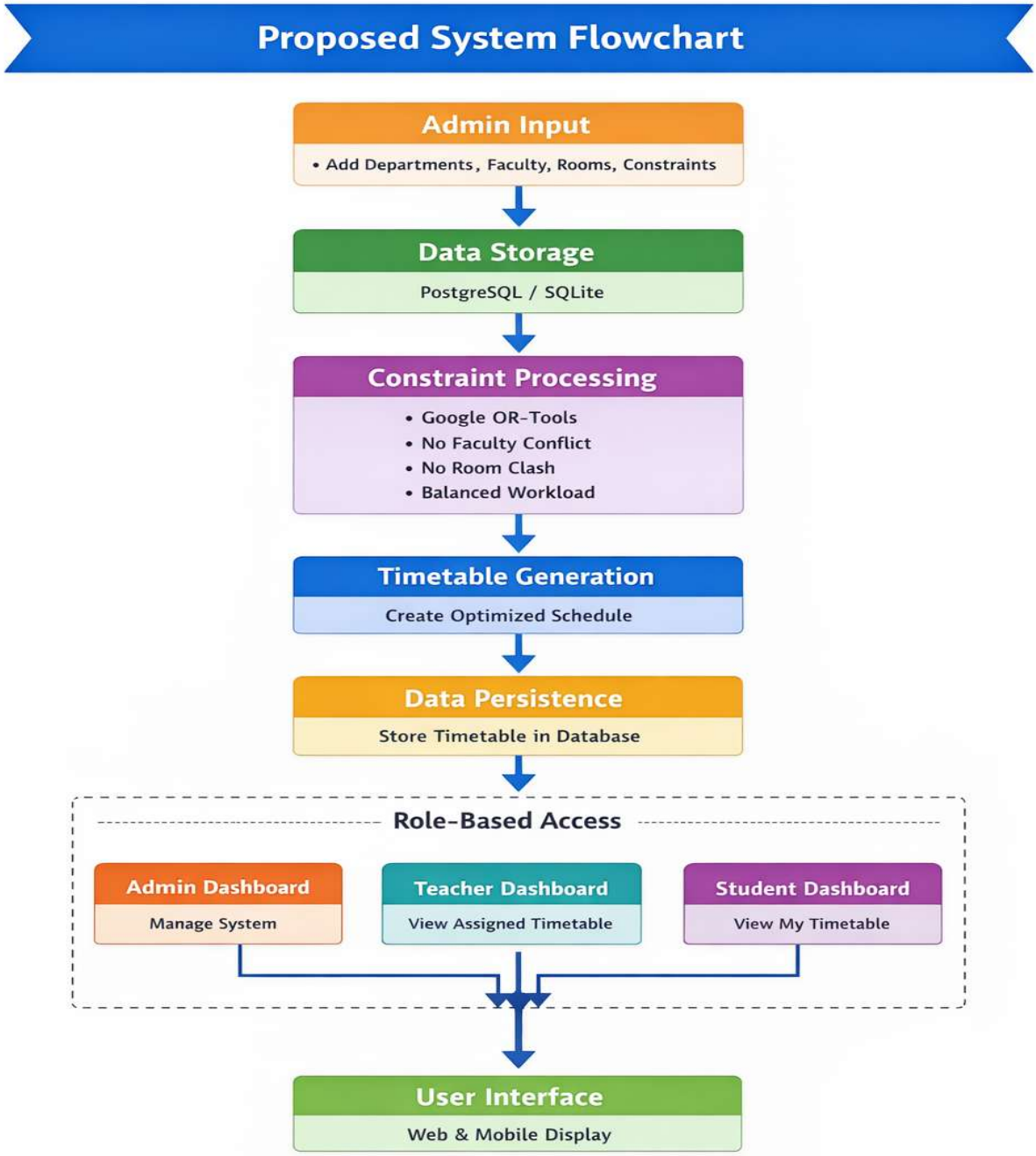
Constraint Processing —► Apply scheduling rules using Google OR-Tools (no clashes, balanced workload, room availability).

Timetable Generation —► Automatically generate optimized timetable based on constraints.

Data Persistence —► Save generated timetable in database.

Role-Based Access Layer —►
- Admin manages system
- Teacher views assigned schedule
- Student views department/year timetable

Display Layer —► Render structured timetable grid in web/mobile interface.


Proposed System Flowchart
Admin Input
• Add Departments, Faculty, Rooms, Constraints
Data Storage
PostgreSQL / SQLite
Constraint Processing
• Google OR-Tools
• No Faculty Conflict
• No Room Clash
• Balanced Workload
Timetable Generation
Create Optimized Schedule
Data Persistence
Store Timetable in Database
Role-Based Access
Admin Dashboard
Manage System
Teacher Dashboard
View Assigned Timetable
Student Dashboard
View My Timetable
User Interface
Web & Mobile Display

d) System Overview

The EduScheduler system consists of the following core modules:

User Management Module
Handles registration, login, and role-based authentication using JWT (Admin, Teacher, Student).

Academic Data Module
Manages departments, faculty, subjects, rooms, and academic years using relational database design.

Scheduling Engine (Constraint Solver)
Uses Google OR-Tools to generate conflict-free timetables based on defined constraints.

Conflict Detection Module
Ensures no faculty double-booking, room clashes, or section overlaps occur.

Timetable Visualization Module
Displays dynamic weekly timetable grids in a clean and user-friendly interface.

Backend Logic & Routing Layer
Flask-based backend manages API requests, database operations, authentication, and scheduling logic.

CSV Management Module
Supports bulk import/export of academic data and timetable backups.

e)Target Users:

Administrators — Manage faculty, departments, and generate optimized timetables efficiently.

Teachers — Access assigned schedules and monitor workload distribution.

Students — View structured timetables filtered by department, year, and section.

Educational Institutions — Improve scheduling efficiency and reduce manual errors.

f)Main Challenges

- Designing a constraint-based scheduling system that ensures zero conflicts across faculty, rooms, and sections
- Managing relational data efficiently while maintaining performance during timetable generation.
- Handling scalability when multiple departments and academic years are involved

SCOPE

- **Current Scope:**

I Automated timetable generation using constraint-based optimization (Google OR-Tools).

I Conflict-free scheduling ensuring no faculty, room, or section overlaps.

I Role-based authentication system (Admin, Teacher, Student) using JWT.

I Relational database management using PostgreSQL/SQLite for structured academic data storage.

I Dynamic timetable visualization through a clean and responsive web/mobile interface.

I Department, year, and section-based filtering for easy timetable access.

I CSV import/export functionality for bulk data handling and backup.

I Secure backend architecture deployed on cloud infrastructure (Render).

I Centralized admin dashboard for managing departments, faculty, subjects, and schedule generation.

.

- **Future Enhancements:**

H AI-Powered Schedule Optimization — Use intelligent algorithms to balance faculty workload and optimize time utilization further.

H Automatic Rescheduling System — Dynamically adjust timetable in case of faculty leave or room unavailability.

H Attendance Management Integration — Enable teachers to mark attendance directly from their timetable.

H Push Notification System — Notify students and teachers about timetable changes in real-time.

H Advanced Analytics Dashboard — Provide workload distribution graphs and scheduling efficiency reports.

H Multi-Institution Support — Extend system to support multiple colleges or campuses under one platform.

H Cloud-Based Scaling — Fully migrate to managed cloud infrastructure with high availability and monitoring.

SIGNIFICANCE OF PROJECT

1. Improved Academic Scheduling Efficiency

•Automated Timetable Generation
EduScheduler eliminates the need for manual timetable preparation by using constraint-based optimization techniques to automatically generate conflict-free schedules. This significantly reduces administrative workload and ensures optimal allocation of faculty, classrooms, and academic resources.

• Faster Schedule Management
The system streamlines the entire scheduling process, enabling administrators to plan, modify, and finalize timetables more efficiently. Changes in faculty availability, room allocation, or subject requirements can be updated dynamically without recreating the entire schedule.

2. Enhanced User Experience

• Role-Based Access System
The platform provides secure and personalized dashboards for administrators, teachers, and students. Each user role has controlled access to relevant information, ensuring privacy, clarity, and efficient system usage.

• Clean & Structured Interface
Timetables are displayed in an organized grid format, making them easy to read and understand. The intuitive interface improves usability and ensures smooth navigation across web and mobile platforms.

• Accessible Anytime, Anywhere
With cloud deployment support, users can access updated schedules remotely, improving convenience and flexibility.

3. Reduction of Human Errors

• Conflict-Free Scheduling
Through automated constraint validation using Google OR-Tools, the system prevents common scheduling errors such as faculty double-booking, classroom clashes, and section overlaps.

• Accurate Data Processing
The backend-driven scheduling engine ensures consistent and reliable timetable generation. Automated validation minimizes manual intervention and significantly reduces the possibility of human mistakes.

4. Data Integrity & Security

• Robust Backend Data Management
The system ensures proper handling, storage, and retrieval of academic data using a relational database

(PostgreSQL/SQLite), maintaining data consistency and preventing data loss.

• Modular and Maintainable Codebase
The application is built using modern development practices, making it scalable, secure, and easy to maintain or enhance in the future.

• Secure Authentication System
JWT-based authentication ensures secure login and role-based authorization, protecting sensitive academic data from unauthorized access.

5. Centralized Monitoring & Management

• Admin Dashboard Control
The system provides centralized management of departments, faculty, subjects, rooms, and scheduling constraints through a dedicated admin dashboard.

• Real-Time Schedule Access
Teachers and students can instantly access updated timetables, ensuring transparency and improved academic coordination.

• Improved Institutional Productivity
By automating scheduling and reducing manual workload, the system enhances overall institutional efficiency and operational effectiveness.

..

TOOLS AND TECNOLOGY USED

Category	Technology Used
Frontend	HTML, CSS, JavaScript, React.js, React native
Backend	Python (Flask)
Database	Sqlite,postgresql
Data Source / API	RESTful APIs (Flask-based)
Data handling	Pandas (for structured data manipulation)
Templating Engine	Jinja2 (for rendering dynamic HTML pages)
Deployment	Vercel (Free Tier), Render (Cloud Hosting)
Version	Git, GitHub
File Handling / Backup	CSV Import & Export
Package Management (Frontend)	npm
ORM (Database Layer)	SQLAlchemy
Authentication	Jwt (java web token)
Scheduling Engine	Google OR-Tools (Constraint Programming)

TIME FRAME REQUIRED FOR VARIOUS STAGES OF PROJECT IMPLEMENTATION

Sr. No.	PHASES	TIME DURATION
1.	Software Requirement Specification	2 weeks
2.	System Design	3 weeks
3.	Coding	4 weeks
4.	Implementation	3 weeks
5.	Testing	2 weeks

REFERENCES

1. Flask Documentation. (2024). *Flask Web Framework*. Available at: https://flask.palletsprojects.com/

2. SQLAlchemy Documentation. (2024). *SQLAlchemy ORM*. Available at: https://docs.sqlalchemy.org/

3. PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. Available at: https://www.postgresql.org/docs/

4. SQLite Documentation. (2024). *SQLite Official Documentation*. Available at: https://www.sqlite.org/docs.html

5. Google Developers. (2024). *OR-Tools Documentation*. Available at: https://developers.google.com/optimization

6. JWT.io. (2024). *JSON Web Token Introduction*. Available at: https://jwt.io/introduction

7. Render Documentation. (2024). *Deploying Python Applications*. Available at: https://render.com/docs

8. React Native Documentation. (2024). *React Native Official Docs*. Available at: https://reactnative.dev/docs/getting-started

9. Cormen, T. H., et al. (2009). *Introduction to Algorithms*. MIT Press. (For algorithmic concepts and optimization principles.)